

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)

Факультет **Прикладной информатики**

Образовательная программа **Мобильные и сетевые технологии**

Направление подготовки(специальность) **09.03.03 Прикладная информатика**

ЛАБОРАТОРНАЯ РАБОТА №5

По дисциплине «Проектирование и реализация баз данных»

Тема: процедуры, функции, триггеры в PostgreSQL

Выполнила Мкртчян К.Г.; К3241.

Проверили Говорова М.М., Белов А.О.

Дата 18.05.2026

Цель работы:

Овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Практическое задание:

- Создать 3 процедуры для индивидуальной БД согласно варианту
- Создать 7 триггеров по ИЗ.

Ход работы

Процедуры

1. Добавить данные о новом звонке абонента.

```
create or replace procedure add_new_contract_call(  
    c_id call_log.contract_id%type,  
    o_zone_id call_log.originator_zone_id%type,  
    d_zone_id call_log.destination_zone_id%type,  
    p_number call_log.phone_number%type,  
    n_type call_log.number_type%type,  
    is_in call_log.is_incoming%type,  
    t_cost call_log.total_cost%type,  
    dt_start call_log.date_time_start%type,  
    dt_end call_log.date_time_end%type  
)  
  
language plpgsql  
  
as  
  
$$  
  
begin  
  
    insert into call_log (  
        contract_id,  
        originator_zone_id,  
        destination_zone_id,  
        phone_number,  
        number_type,  
        is_incoming,  
        total_cost,  
        date_time_start,  
        date_time_end  
    )  
    values (  
        c_id,  
        o_zone_id,  
        d_zone_id,  
        p_number,  
        n_type,  
        is_in,  
        t_cost,  
        dt_start,  
        dt_end  
    );  
  
end;
```

```

        is_incoming,
        total_cost,
        date_time_start,
        date_time_end
    )
values (
    c_id,
    o_zone_id,
    d_zone_id,
    p_number,
    n_type,
    is_in,
    t_cost,
    dt_start,
    dt_end
);
end;
$$;

```

Вызовем процедуру и проверим результат ее работы:

```

call add_new_contract_call(
    1930,
    234,
    235,
    '+7-976-270-8571' :: character(15),
    'мобильный' :: character varying,
    true,
    12.50 :: money,

```

```

now(),
now() + interval '5 minutes'
);

```

```

select contract_id, originator_zone_id, destination_zone_id,
phone_number, date_time_start from call_log
where contract_id = 1930
order by date_time_start desc;

```

В результате запроса видим запись с недавним звонком нашего абонента:

	contract_id integer	originator_zone_id integer	destination_zone_id integer	phone_number character (15)	date_time_start timestamp with time zone
1	1930	234	235	+7-976-270-85...	2026-05-22 14:35:51.318828+...
2	1930	216	224	+7-945-568-56...	2026-03-06 02:11:04+03

2. Вывести задолженность по оплате для заданного абонента

```

create or replace procedure get_debtors(
    inout ref refcursor,
    c_id integer
)
language plpgsql
as
$$
begin
    open ref for
        select contract_id, least(balance, 0::money) as debt from contracts
        where contract_id = c_id;
end;

```

\$\$;

Выполним последовательно команды:

```
begin;
```

```
call get_debtors('mycursor'::refcursor, 1932 :: integer);
```

```
fetch all from mycursor;
```

```
commit;
```

Получим результат с долгом абонента:

	contract_id [PK] integer	debt money
1	1931	-5 598,0...

Если у абонента положительный баланс, то долга у него нет:

	contract_id [PK] integer	debt money
1	1932	0,00 ?

3. Рассчитать общую стоимость звонков по каждой зоне за истекшую неделю

```
create or replace procedure get_week_zones_total_cost(
```

```
    inout ref refcursor
```

```
)
```

```
language plpgsql
```

```
as
```

```
$$
```

```
begin
```

```
    open ref for
```

```
    select
```

```
        originator_zone_id,
```

```

        sum(total_cost) as total_cost
from call_log
where date_time_start >= NOW() - interval '7 days'
group by originator_zone_id;
end;
$$;

```

Выполним последовательно команды:

```

begin;
call get_week_zones_total_cost('mycursor'::refcursor);
fetch all from mycursor;
commit;

```

Получим результат с общей стоимостью звонков по каждой зоне за истекшую неделю:

	originator_zone_id  integer	total_cost  money
1	220	430,00 ?
2	234	155,00 ?

Триггеры

1. Триггер на обновление баланса при внесении средств

```
create or replace function update_balance_on_deposit()
```

```
returns trigger as $$
```

```
begin
```

```
    update contracts
```

```
    set balance = balance + new.amount
```

```
    where contract_id = new.contract_id;
```

```
    return new;
```

```
end;
```

```
$$ language plpgsql;
```

```
create trigger trg_update_balance_on_deposit
```

```
after insert on deposits
```

```
for each row
```

```
    execute function update_balance_on_deposit();
```

Выполним запросы для проверки работы триггера:

```
insert into deposits (contract_id, amount, method, date_time)
```

```
    values (1914, 300, 'смс-команда', now());
```

```
—
```

```
select * from contracts
```

```
    where contract_id = 1914;
```


Результаты:

	contract_id [PK] integer	balance money
1	1914	8 321,00 ?

	contract_id [PK] integer	balance money
1	1914	8 621,00 ?

2. Триггер на отключение не актуального тарифа у пользователя

create or replace function deactivate_connected_tariff()

returns trigger as \$\$

begin

update connected_tariff

set disconnection_date = new.connection_date

where contract_id = new.contract_id and disconnection_date is null

and tariff_connection_id <> new.tariff_connection_id;

return new;

end;

\$\$ language plpgsql;

create or replace trigger trg_deactivate_connected_tariff

before insert on connected_tariff

for each row

execute function deactivate_connected_tariff();

Выполним запросы для проверки работы триггера:

insert into connected_tariff (contract_id, tariff_id, connection_date)

```
values (1845, 604, now());
```

—

```
select * from connected_tariff
```

```
where contract_id = 1845;
```

Результаты:

	tariff_connection_id [PK] integer	contract_id integer	tariff_id integer	connection_date timestamp with time zone	disconnection_date timestamp with time zone
1	1695	1845	620	2026-01-10 12:48:35+03	[null]

	tariff_connection_id [PK] integer	contract_id integer	tariff_id integer	connection_date timestamp with time zone	disconnection_date timestamp with time zone
1	1801	1845	604	2026-05-20 21:59:49.224522+...	[null]
2	1695	1845	620	2026-01-10 12:48:35+03	2026-05-20 00:00:00+03

3. Триггер на списание абонентской платы при подключении/смене тарифа

```
create or replace function connect_or_charge_tariff()
```

```
returns trigger as $$
```

```
declare
```

```
    tariff_price money;
```

```
begin
```

```
-- Получаем актуальную цену тарифа
```

```
select price into tariff_price
```

```
from tariffs_price
```

```
where tariff_id = new.tariff_id
```

```
    and start_date <= current_date
```

```
    and (expiration_date is null or expiration_date >= current_date)
```

```
order by start_date desc
```

```
limit 1;
```

```
-- Списание
```

```

insert into tariff_debitings (tariff_connection_id, date_time, amount)
values (new.tariff_connection_id, now(), tariff_price);

-- Уменьшаем баланс договора

update contracts
set balance = balance - tariff_price
where contract_id = new.contract_id;

return new;

end;

$$ language plpgsql;

create or replace trigger trg_connect_or_charge_tariff
after insert on connected_tariff
for each row
execute function connect_or_charge_tariff();

```

Обновили тариф:

```

select * from connected_tariff
where contract_id = 1850;

```

	tariff_connection_id [PK] integer	contract_id integer	tariff_id integer	connection_date timestamp with time zone	disconnection_date timestamp with time zone
1	1700	1850	609	2026-01-04 05:08:14+03	[null]

	tariff_connection_id [PK] integer	contract_id integer	tariff_id integer	connection_date timestamp with time zone	disconnection_date timestamp with time zone
1	1700	1850	609	2026-01-04 05:08:14+03	2026-05-20 23:14:11.822696+...
2	1815	1850	604	2026-05-20 23:14:11.822696+...	[null]

Появилась новая запись в tariff_debitings:

```
select * from tariff_debitings
```

```
where tariff_connection_id in (select tariff_connection_id from connected_tariff where  
contract_id = 1850);
```

	debiting_id [PK] integer	tariff_connection_id integer	amount money	date_time timestamp with time zone
1	2097	1700	220,00 ?	2026-03-27 20:42:16+03
2	2102	1700	343,00 ?	2026-01-07 00:14:41+03
3	2109	1700	399,00 ?	2026-01-27 17:59:48+03
4	2110	1700	77,00 ?	2026-02-10 19:00:37+03

4	2110	1700	77,00 ?	2026-02-10 19:00:37+03
5	2207	1815	500,00 ?	2026-05-20 23:14:11.822696+...

Обновился баланс абонента:

```
select contract_id, balance from contracts
```

```
where contract_id = 1850;
```

	contract_id [PK] integer	balance money
1	1850	7 037,00 ?

	contract_id [PK] integer	balance money
1	1850	6 537,00 ?

4. Триггер на деактивацию не актуальной цены тарифа

```
create or replace function deactivate_tariff_price()
```

```
returns trigger as $$
```

```
begin
```

```
    update tariffs_price
```

```
        set expiration_date = new.start_date
```

```
where tariff_id = new.tariff_id and (expiration_date is null or expiration_date >
new.start_date);
```

```
return new;
```

```
end;
```

```
$$ language plpgsql;
```

```
create or replace trigger trg_deactivate_tariff_price
```

```
before insert on tariffs_price
```

```
for each row
```

```
execute function deactivate_tariff_price();
```

Запросы для проверки:

```
select * from tariffs_price
```

```
where tariff_id = 604;
```

```
insert into tariffs_price (tariff_id, price, modification_date, start_date)
```

```
values (604, 1000, current_date, current_date);
```

Изменения в таблице tariffs_price:

	current_price_id [PK] integer	tariff_id integer	price money	modification_date date	start_date date	expiration_date date
1	555	604	1 903,0...	2023-01-28	2023-02-07	2023-05-10
2	602	604	500,00 ?	2026-05-20	2026-05-20	[null]

	current_price_id [PK] integer	tariff_id integer	price money	modification_date date	start_date date	expiration_date date
1	555	604	1 903,0...	2023-01-28	2023-02-07	2023-05-10
2	602	604	500,00 ?	2026-05-20	2026-05-20	2026-05-20
3	605	604	1 000,0...	2026-05-20	2026-05-20	[null]

5. Триггер на вычет минут за звонки

create or replace function `handle_call_minutes()`

returns trigger as \$\$

declare

`call_duration` smallint;

`remaining_minutes` smallint;

`active_tariff_id` int;

`overcharge_minutes` smallint;

`minute_price` money;

begin

-- Только для исходящих звонков

if `new.is_incoming` = true then

return new;

end if;

-- Длительность звонка в минутах (округление вверх)

`call_duration` := `ceil(extract(epoch from (new.date_time_end - new.date_time_start)) / 60)::smallint`;

-- Находим активный тариф договора

select `tariff_connection_id` into `active_tariff_id`

```

from connected_tariff

where contract_id = new.contract_id

    and disconnection_date is null

limit 1;

-- Получаем остаток минут по тарифу

select minutes into remaining_minutes

from tariff_balances

where tariff_connection_id = active_tariff_id;

-- Если остаток не найден, создаём запись с нулями

if remaining_minutes is null then

    insert into tariff_balances (tariff_connection_id, internet, sms, minutes)

    values (active_tariff_id, 0, 0, 0);

    remaining_minutes := 0;

end if;

-- Списание минут

if remaining_minutes >= call_duration then

    -- Хватает минут в пакете

    update tariff_balances

    set minutes = minutes - call_duration

    where tariff_connection_id = active_tariff_id;

else

    -- Не хватает минут: списываем остаток пакета

```

```

update tariff_balances

set minutes = 0

where tariff_connection_id = active_tariff_id;

-- Считаем, сколько минут сверх пакета

overcharge_minutes := call_duration - remaining_minutes;

-- Получаем цену минуты для зоны назначения

select price into minute_price

from minute_price

where zone_id = new.destination_zone_id

and start_date <= new.date_time_start

and (expiration_date is null or expiration_date >= new.date_time_start)

limit 1;

-- Списание денег за сверхлимитные минуты

update contracts

set balance = balance - (minute_price * overcharge_minutes)

where contract_id = new.contract_id;

end if;

return new;

end;

$$ language plpgsql;

```


create or replace trigger `trg_handle_call_minutes`

after insert on `call_log`

for each row

execute function `handle_call_minutes()`;

Выполним следующие запросы:

insert into `call_log` (`contract_id`, `originator_zone_id`,

`destination_zone_id`, `phone_number`, `number_type`, `is_incoming`, `total_cost`,
`date_time_start`, `date_time_end`)

values (`1886`, `220`, `211`, `'+7-976-270-8571'`, `'мобильный'`, `false`, `200`, `now()`, `now()` +
`interval '5 minutes 30 seconds'`);

select `contract_id`, `originator_zone_id`, `is_incoming` from `call_log`

where `contract_id` = `1886`;

select `c.contract_id`, `c.balance`, `tb.minutes` from `contracts` as `c`

join `connected_tariff` as `ct` on `c.contract_id` = `ct.contract_id`

join `tariff_balances` as `tb` on `ct.tariff_connection_id` = `tb.tariff_connection_id`

where `c.contract_id` = `1886`;

Таблица `call_log`:

	<code>contract_id</code> integer	<code>originator_zone_id</code> integer	<code>is_incoming</code> boolean
1	1886	235	true
2	1886	211	true
3	1886	220	true

	contract_id integer	originator_zone_id integer	is_incoming boolean
1	1886	235	true
2	1886	211	true
3	1886	220	true
4	1886	220	false

Баланс и минуты абонента:

	contract_id integer	balance money	minutes smallint
1	1886	3 288,00 ?	840

	contract_id integer	balance money	minutes smallint
1	1886	3 288,00 ?	834

6. Триггер на запрет удаления записей

create or replace function `prevent_delete_from_history()`

returns trigger as \$\$

begin

 raise exception 'Удаление записей из таблицы % запрещено', tg_table_name;

 return null;

end;

\$\$ language plpgsql;

-- Для таблицы call_log (журнал звонков)

create trigger `trg_prevent_delete_call_log`

before delete on `call_log`

for each row

execute function `prevent_delete_from_history()`;

-- Для таблицы deposits (платежи)

```
create trigger trg_prevent_delete_deposits
```

```
before delete on deposits
```

```
for each row
```

```
execute function prevent_delete_from_history();
```

-- Для таблицы tariff_debitings (списания)

```
create trigger trg_prevent_delete_tariff_debitings
```

```
before delete on tariff_debitings
```

```
for each row
```

```
execute function prevent_delete_from_history();
```

Без триггера:

```
delete from call_log
```

```
where contract_id = 1930 and originator_zone_id = 234
```

```
select * from call_log
```

```
where contract_id = 1930
```

	call_id [PK] integer	contract_id integer	originator_zone_id integer	destination_zone_id integer	phone_number character (15)	number_type character varying (20)
1	839	1930	234	216	+7-970-498-57...	мобильный
2	1041	1930	239	211	+7-932-878-46...	городской
3	1043	1930	216	224	+7-945-568-56...	мобильный
4	1251	1930	234	235	+7-976-270-85...	мобильный

	call_id [PK] integer	contract_id integer	originator_zone_id integer	destination_zone_id integer	phone_number character (15)	number_type character varying (20)
1	1041	1930	239	211	+7-932-878-46...	городской
2	1043	1930	216	224	+7-945-568-56...	мобильный

После применения триггера:

```
ERROR:  Удаление записей из таблицы call_log запрещено
CONTEXT:  функция PL/pgSQL prevent_delete_from_history(), строка 3, оператор RAISE

ОШИБКА:  Удаление записей из таблицы call_log запрещено
SQL state: P0001
```

7. Триггер на автоматический расчет стоимости звонка

create or replace function `calculate_call_cost()`

returns trigger as \$\$

declare

`duration_minutes` smallint;

`zone_price` money;

begin

-- Длительность в минутах (округление вверх)

`duration_minutes := ceil(extract(epoch from (new.date_time_end -
new.date_time_start)) / 60);`

-- Цена минуты

`select price into zone_price`

`from minute_price`

`where zone_id = new.originator_zone_id`

`and start_date <= new.date_time_start`

`and (expiration_date is null or expiration_date >= new.date_time_start)`

`order by start_date desc`

`limit 1;`

`new.total_cost := zone_price * duration_minutes;`

```

return new;

end;

$$ language plpgsql;

create or replace trigger trg_calculate_call_cost

before insert on call_log

for each row

execute function calculate_call_cost();

```

После применения триггера стоимость звонка посчиталась как $5.00 * 6 = 30.00$

```

select contract_id, originator_zone_id, total_cost, (date_time_end - date_time_start) from
    call_log

where contract_id = 1886

order by date_time_start desc;

—

insert into minute_price (zone_id, price, modification_date, start_date)

values (220, 5.00, current_date, current_date);

—

insert into call_log (contract_id, originator_zone_id,

destination_zone_id, phone_number, number_type, is_incoming, total_cost,

date_time_start, date_time_end)

values (1886, 220, 211, '+7-976-270-8571', 'мобильный', false, 0, now(), now() + interval

'5 minutes 30 seconds');

```

	contract_id  integer	originator_zone_id  integer	total_cost  money	?column? interval
1	1886	220	30,00 ?	00:05:30
2	1886	220	200,00 ?	00:05:30

Выводы

В ходе работы была спроектирована инфологическая модель БД «Телефонный провайдер», включающая 22 сущности. Разработаны и реализованы триггеры для автоматизации основных процессов: обновление баланса, контроль тарифов, списание минут и средств за звонки, расчёт стоимости звонков, деактивация устаревших записей. Выполнена проверка работоспособности триггеров тестовыми запросами – все функции работают корректно.